Appendix J2 — More questions for your work-environment Claude — telemetry, metrics, and the distributed-systems layer of your projects

Source: `OFFICE_CLAUDE_QUESTIONS_DISTRIBUTED_SYSTEMS.md`

Office-Claude Questions, round 2 — telemetry, metrics, and the distributed-systems layer of YOUR projects

What this adds. `OFFICE_CLAUDE_QUESTIONS.md` (infra/deploy/CI-CD/K8s/Data Cloud) and `salesforce_architecture_probe.md` (architecture, evals, permissions) are already written. This file covers what they don't: **telemetry and metrics**, and the **distributed-systems topics** a deep-tech interviewer (Karan Bhatia: Azure AD identity, New Relic multi-tenancy/observability) will aim at *your* projects — data modeling, design, contention, multi-step processes, orchestration vs choreography, scaling the DB, sharding, caching.

The projects these questions are about (replace with the exact names your team uses):

- **P1** Agentforce for Setup v1→v2 contributions (planner/knowledge/executor, NLDs)
- **P2** The five access-management workflows (~60 days → ~5 days)
- **P3** The Slack access-management agent (Python + TypeScript, Bolt)
- **P4** The AI evaluation framework
- **P5** Headless 360 — the eight onboarded capabilities / MCP
- **P6** AI Bug Triage skill

How to run this (read once)

- One section per session. Paste the **preamble** first every time.
- You are collecting **truth underneath** — for you. In the interview you speak at the **pattern level** with rounded or relative numbers. **Follow your company's confidentiality rules:** don't copy internal names/numbers/links/screenshots into personal files or an external tool. Write *your own* paraphrase: "queue-based, per-workflow workers, ~minutes p95", not the internal service names.
- After each session fill the **defense card** (last section) for that project.
- Anything the answer can't evidence → mark **THIN** and prepare to say "platform-owned; here's how I'd reason about it."

Preamble (paste first):

> I'm preparing to explain my own work precisely in a technical interview. For every answer: (1) tag each claim **VERIFIED (cite the repo path / doc / dashboard / PR / incident)** or **INFERRED (reasoning only)** — never blend them. (2) Tell me separately what **I personally built/owned**, what I **partly contributed to**, and what I only **consumed as a platform capability** — use commit/PR/ownership data where you can see it. (3) If you can't find evidence, say "not found" rather than guessing. (4) Give numbers as ranges or relative terms I can safely say at the pattern level. (5) End with the 3 questions an expert reviewer would ask next.

T. Telemetry and metrics — what is actually measured, and how I'd prove my numbers

> **Why this section matters most for Karan:** observability is his stated expertise and his published vocabulary (`traceId`, `upstreamApplication`, `tenantId/userId` correlation attributes). If you can say exactly what your agents emit and what you alert on, you're on his ground.

T1 — What does each project emit, per request? For P1/P2/P3/P5, list every span/log/metric emitted along one request path (entry → triage → retrieval → plan compile → validation → approval → execute → verify): name, type (span/log/counter/histogram/gauge), the attributes carried (is there a correlation/trace id? a tenant/org id? user id? flow/workflow name? node name? tool name? plan id? NLD/skill version? model/version?), the sink (Splunk? another APM? Data Cloud? an internal metrics store?), and retention. **Is the trace id propagated across every hop — including the async/queue hop and the Slack hop? Where does it break?**

T2 — The golden-signal dashboard. For each project: what are the **SLIs** (latency percentiles per stage, error rate by class, throughput, saturation: queue depth/worker utilization), the **SLOs** if any, and the alerts that page someone. What's the *actual* dashboard I'd open at 3 a.m.? Which 3 panels?

T3 — Agent-specific metrics. Which of these exist, and how are they computed? Triage accuracy / misroute rate · retrieval hit rate (recall@k, MRR) and score-threshold drop rate · funnel shortlist hit rate (did the right action appear in the top 3–5?) · plan validation reject rate by gate (schema / allowlist / semantic / policy) · clarification rate (`GatherMoreDetails` fired) · approval rate / rejection rate / edit rate / approval latency (p50/p90) · execution success rate, retry rate, partial-failure rate · read-back mismatch rate · cost per request (tokens in/out, model, retrieval calls) · time-to-resolution end to end · cache hit rate · human-escalation rate · per-tenant breakdown of all of the above.

T4 — Prove my resume numbers from telemetry. (a) "~60 days → ~5 days (~92%)" — what is the *start event* and *end event*, which system holds those timestamps, over what window and how many requests, median or mean, and p90? What else changed in that window (team size, request mix, a policy change)? (b) "~20 min → under 3 min" (Slack) — same questions; what is the 3 minutes made of (LLM, grounding, **human approval wait**, execution)? (c) "~35% fewer misconfigured requests" — exact definition of *misconfigured*, numerator/denominator, before/ after windows, relative vs absolute. (d) "Throughput ~40% up" — unit (executions/hour at the same pod count?), load, how it was measured. (e) "150+ users, 8 teams" — how counted (distinct Slack users? active in window?). For each: the query/dashboard/report that produced it (describe it in words), and its honest weakness.

T5 — One incident, reconstructed from telemetry. Pick a real production incident in P1–P5. Give me the timeline: alert → what you queried → what the trace showed → root cause → fix → what telemetry you added afterwards. Include the exact shape of the query (fields filtered/grouped, not internal index names).

T6 — Online evals. Do the same scorers that run in CI also run on sampled production traces? What's sampled, what's scored, where do results land, what alerts on them? What's the false-positive rate of the alert?

T7 — Gaps. What *can't* I currently see? (Per-tenant cost? Retrieval staleness? Approver behavior? Cross-service trace continuity?) What would I add first and why?

---

1. Data modeling

D1 — Entities. For P1/P2/P3 list the core persisted entities and their keys/relationships: request/ session/conversation, plan (`executionPlan`), plan step, approval, execution result, audit record, NLD/Skill metadata + version, mDLO/DMO rows for knowledge, eval case/result, feedback. For each: primary key, how it is identified across systems (is there one **correlation id**?), tenant scoping column, lifecycle states.

D2 — State machines. Draw the state machine of a request and of a plan (e.g., `RECEIVED → TRIAGED → CLARIFYING → PLANNED → VALIDATED → AWAITING_APPROVAL → EXECUTING → VERIFIED | FAILED | COMPENSATED`).

Which transitions are guarded by **conditional updates** (compare-and-set on status)? Which states are terminal? Where is the state stored?

D3 — Immutability and audit. What is append-only (audit/ledger/event log) vs mutable? Can I reconstruct "why did the agent do X" from stored data alone? What's the retention and the access control?

D4 — Versioning. How are NLDs/Skills, prompts, tool contracts, and plans versioned? Does every run record which versions it used? How is rollback done?

D5 — Multi-tenancy in the model. How is tenant identity carried on every record and every index? Is isolation by row filter, by namespace/dataspace/partition, or by separate storage? What prevents a missing tenant filter from leaking data?

D6 — Schema evolution. How do you change a plan/tool schema without breaking in-flight (paused) runs?

---

2. System design and boundaries

S1 — Component and ownership map for P1–P5: each service/component, what it owns, its API (sync/async), who calls it, and which team owns it. Mark mine. **S2 — Sync vs async.** Which calls are request/response and which go through queues/events? Where does the **3-second Slack ack** boundary sit, and what happens after it? **S3 — Failure domains.** If component X is down, what degrades and what stops? Where are the **timeouts, retries, circuit breakers, bulkheads**? Any single points of failure? **S4 — Design decisions with alternatives.** The 3 biggest build-vs-buy / design decisions I was part of, the alternatives rejected, the evidence used, and what I'd change now.

---

3. Data contention and concurrency (your "Dealing with Contention" answers)

C1 — Concurrent actors. What happens if **two admins** request changes to the **same user/permission set** at once? Two plans for one entity? A human edits the record in Setup while the agent's plan is awaiting approval? Show the actual mechanism (optimistic version check, record locking, conditional update, platform transaction semantics) — or confirm there is none. **C2 — Double submit.** Double-clicked approval button? Slack redelivery of an un-acked event? Retried execute after a timeout? What is the **idempotency key** and where is it enforced (unique constraint? store? TTL)? Is "re-assign an already-assigned permission set" a no-op or an error? **C3 — Time-of-check vs time-of-use.** Between plan-time grounding and execute-time, state can change (user deprovisioned, permission revoked, entity renamed). What **re-validation** happens at execution? Fail closed? **C4 — Platform limits as contention.** Which **governor limits / API rate limits / concurrency limits** bite (per-org API calls, DML limits, row-lock errors like `UNABLE_TO_LOCK_ROW`)? How does the executor retry, back off, and bound attempts? **C5 — Isolation level.** For any store I touch (Core DB, Data Cloud, a Postgres/state store for runs), what isolation level/semantics apply, and have we hit a lost update or write-skew? Real example? **C6 — Hot spots.** Any hot record/tenant/queue? What was done (sharded counter, batching, queue partitioning, per-tenant rate limit)?

---

4. Multi-step processes

M1 — The longest workflow. Walk P2's longest workflow step by step: each step, its side effect, whether it's **idempotent**, its timeout, its retry policy, what's persisted after it. **M2 — Partial failure.** If step 3 of 5 fails after step 2 committed, what exactly happens? Compensating action? Forward-fix? Manual remediation? Is there a "pivot" step after which you only go forward? Real example. **M3 — Pause/resume.** How does a paused run (clarification, approval) persist across minutes/hours/restarts? What is stored, where, with what TTL; what happens when it expires; **can it be resumed twice? M4 — Long waits.** Approval latency of hours/days: token lifetime, session expiry, plan staleness — handled how? **M5 — Saga/outbox patterns.** Do we write to a DB **and** publish an event anywhere (dual-write)? Is there an outbox/CDC? How is "exactly-once effect" achieved (idempotent consumers)?

---

5. Orchestration vs choreography

O1 — Which is it, where? For P1 (supervisor + specialists = orchestration inside the agent) and for the cross-service flow around it (Slack → agent → platform APIs → notification), identify which parts are **orchestrated** (one component drives the steps and compensations) and which are **choreographed** (services react to events). Who owns the end-to-end flow definition? Where would I look to see the whole flow? **O2 — Why that choice**, what alternative was considered (a workflow engine like Step Functions/Temporal/ internal equivalents vs. in-process graph), and what the downsides showed up in production (debuggability, coupling, ordering). **O3 — Visibility.** How do you answer "where is request X right now?" Is there a single place?

---

6. Distributed-systems mechanics

X1 — Delivery semantics. For each hop (Slack → service, service → queue, queue → worker, worker → platform API, retrieval ingestion pipeline): at-most/at-least/exactly-once *in practice*? Where's dedupe? **X2 — Ordering.** Anywhere ordering matters (events for one user/tenant)? How preserved (partition key, single consumer)? What if violated? **X3 — Consistency model.** For each store/pipeline: strong, read-your-writes, or eventual? Concrete lag numbers: ingestion → searchable, permission change → visible to the retriever. A **real staleness incident** (answer was confidently wrong because an index lagged)? **X4 — Retries and backoff.** Policy per call: attempts, backoff, jitter, what is retried vs surfaced; any retry storm / thundering herd ever? **X5 — Timeouts and budgets.** End-to-end latency budget and per-stage timeouts; what happens at the LLM call timeout; **step/token/cost budgets** that bound a runaway loop. **X6 — Backpressure and rate limiting.** Where does the system say "slow down"? Per-tenant fairness? Load shedding? **X7 — Clocks and TTLs.** Anything depending on time (token expiry, lease TTLs, idempotency windows)? Any clock-skew issue?

---

7. Scaling the database / data stores

B1 — What stores does each project actually use? (relational Core DB? Data Cloud lakehouse? a vector/ search index? a key-value/state store? object storage? a cache?) For each: engine, what's stored, access pattern (point read / scan / vector search), size order-of-magnitude, growth, read:write ratio. **B2 — Bottlenecks hit.** Slow queries, index changes, connection limits, replication lag — a real one with the fix (index added? query rewritten? caching? denormalization?). **B3 — Retrieval scaling.** The HLD calls centralized retrieval scalability "uncertain" — what's the current status, measured p50/p95, and the plan? Shared index + tenant filter vs partitions? **B4 — Ingestion scaling.** Metadata-extraction/mDLO pipeline: throughput, queueing, the stuck-state failures (`Syncing`, `Ingestion_Requested`), end-to-end duration for a full org. **B5 — Capacity numbers.** Requests/day, peak QPS, tokens/request, stored objects/tenant — rough order.

---

8. Sharding, partitioning, tenancy

H1 — Unit of isolation. Is the tenant unit an org/instance/pod? How are tenants **placed** on infrastructure? Is there a routing layer/directory? What happens when a tenant is huge (noisy neighbor)? **H2 — Partitioning of knowledge/vector data.** Per-tenant dataspace/index vs shared + filter; how are partitions chosen; resharding/rebalancing ever done? **H3 — Queue/worker partitioning.** Partition key for work queues; per-workflow bulkheads; how ordering and fairness are handled. **H4 — Cross-partition operations.** Anything that spans tenants/partitions (global eval runs, analytics) and how it avoids leaking data.

---

9. Caching

K1 — What is cached, where, and for how long? (NLDs served from Core's classpath, retrieval results, metadata grounding, tokens/orgJWT, embeddings, plan results). For each: key, TTL, invalidation trigger, hit rate, size. **K2 — Invalidation and staleness.** How does a changed NLD/permission/metadata invalidate caches? Any **stale-cache**

incident? Is any cache ever the source of truth for an authorization decision (it shouldn't be)? **K3 — Failure behavior.** Stampede on expiry (many requests, one key)? Cache outage → fallback or outage? Negative caching? **K4 — Token caching.** orgJWT / user tokens: TTL vs approval latency; refresh; revocation propagation.

---

10. Security-and-identity checks that tie these together (his home turf)

I1 — The identity chain for P3 and P1, hop by hop: what credential exists at each hop, who minted it, audience, scope, TTL, how it's narrowed, and where the **requester's own permissions** are re-checked. **I2 — Revocation.** User deprovisioned mid-flow: what actually happens, and how fast do caches/tokens/ indexes reflect it? **I3 — Audit.** Can I answer "who approved, what exactly changed, under whose authority, when, why" from stored records?

Per-project DEFENSE CARD — fill one per project (this is the deliverable)

```
PROJECT: _____   MY ROLE: OWN / PARTIAL / CONSUMED   (evidence: PR/doc/dashboard)
ONE-LINE WHAT & WHY:
FLOW (8 beats): trigger+authority → route → ground → plan → gate → act → verify+audit → learn+operate
DATA MODEL: entities · keys · states · tenant column · versioning
CONCURRENCY: contention case · mechanism · idempotency key · re-validation at execute
MULTI-STEP: longest workflow · partial-failure behavior · pause/resume storage
ORCH vs CHOREO: what's orchestrated · what's event-driven · who owns the whole flow
DISTRIBUTED: delivery semantics · ordering · consistency lag (number) · retry policy
SCALE: stores · bottleneck hit & fix · partitioning/tenant isolation · cache + invalidation
TELEMETRY: 3 dashboard panels · 3 alerts · trace-id continuity · the metric behind my headline number
ONE REAL INCIDENT: timeline · root cause · fix · what we instrumented after
HONEST WEAKNESS: one thing that's not great, and what I'd do next
THIN / PLATFORM-OWNED (say so): _____
```

After each session

1. Paste only your **paraphrased, pattern-level** answers into the defense card (not internal names/links).
2. Update the **ownership table** (Part II §15.15 of the master PDF) with OWN/PARTIAL/CONSUMED + one citation.
3. Add every number you can defend to the §2.1 five-question template; **soften any you can't.**
4. Drop anything you can't evidence — "I don't know that; here's how I'd reason" beats a bluff.
